



# APLICACIONES MÓVILES

DOCENTE: GABRIEL EDUARDO MOREJÓN  
LÓPEZ

## ACTIVIDAD 10: PRÁCTICAS

UNIDAD 3

**GREGORIO JAZMANI COBEÑA INTRIAGO**  
**NOHELIA CAROLINA LEON GILER**  
**ROBERTO EDUARDO MORAN RAMIREZ**  
**JORDAN FREDDY PONCE WILA**  
**JOEL ALEXANDER SANTILLAN ZAMBRANO**

8 DE JULY DE 2026

## UNIDAD 3

### 1. INTRODUCCIÓN

El desarrollo de aplicaciones móviles se ha convertido en una habilidad central en la educación en ingeniería de software porque permite la transferencia de conceptos de programación, diseño de interfaces y gestión de datos a productos que el usuario final puede utilizar directamente desde su dispositivo. Dentro del ecosistema de herramientas para este fin, MIT App Inventor destaca como una plataforma educativa que simplifica la creación de aplicaciones a través de un enfoque visual basado en bloques, eliminando gran parte de la barrera sintáctica de los lenguajes de programación tradicionales. El propósito de este informe es explorar y documentar aspectos clave de MIT App Inventor: su definición y arquitectura general, dos entornos de trabajo (Designer y Blocks), el componente de almacenamiento persistente de TinyDB, la herramienta de pruebas en vivo AI Companion y sus ventajas y limitaciones frente a otras alternativas de desarrollo. Finalmente, se presenta una comparación estructurada entre MIT App Inventor y el framework Ionic, contrastando aspectos como el lenguaje utilizado, la curva de aprendizaje, el rendimiento y el proceso de lanzamiento de la aplicación.

### 2. ¿QUÉ ES MIT APP INVENTOR Y SU ARQUITECTURA?

MIT App Inventor es un entorno de desarrollo visual gratuito basado en la nube creado originalmente por Google Labs y actualmente mantenido por el Instituto de Tecnología de Massachusetts (MIT). Permite a cualquier persona, incluso sin experiencia previa en programación, crear aplicaciones Android e iOS completamente funcionales ensamblando bloques visuales en lugar de escribir código de texto. La plataforma se basa en un concepto de "piso bajo, techo alto", es decir, ofrece una introducción sencilla para principiantes, pero permite la creación de aplicaciones de considerable complejidad a medida que el usuario avanza. Desarrollado originalmente por el profesor Hal Abelson con el equipo de Google Education, en 2013 el MIT lanzó la versión 2 de App Inventor, que introdujo el uso de Blockly (la biblioteca de Google para crear lenguajes de programación visual) como motor de edición de bloques.



#### 2.1 ARQUITECTURA GENERAL

La arquitectura de MIT App Inventor se organiza en tres grandes componentes que trabajan de manera conjunta: el servidor en la nube (App Inventor Server), que aloja el entorno, gestiona los proyectos y compila el código de bloques en paquetes APK o AAB mediante un servicio remoto basado en Java; el editor web (cliente), una interfaz de navegador dividida en las vistas Designer y Blocks para diseñar la interfaz y definir el comportamiento; y el dispositivo o emulador de prueba, que permite visualizar cambios en tiempo real mediante MIT AI2 Companion, USB o un emulador. Internamente, cada bloque visual se traduce a código Java/Kotlin sobre el framework de componentes de Android durante la compilación, generando una aplicación nativa que, en sus versiones más recientes, también ofrece compatibilidad experimental con iOS a través de Skia y CocoaPods.

## Arquitectura general de MIT App Inventor



*Los tres componentes se comunican de forma continua: el navegador edita el proyecto, el servidor compila y el dispositivo ejecuta y refleja los cambios en tiempo real ("live testing").*

### 3. ENTORNO DESIGNER

El Diseñador es la primera de dos vistas de trabajo en App Inventor y corresponde a la fase de creación visual de la interfaz de usuario. En esta vista, el desarrollador arrastra y suelta componentes prediseñados desde una paleta organizada por categorías en el área de la pantalla (visualización) que se va a editar.

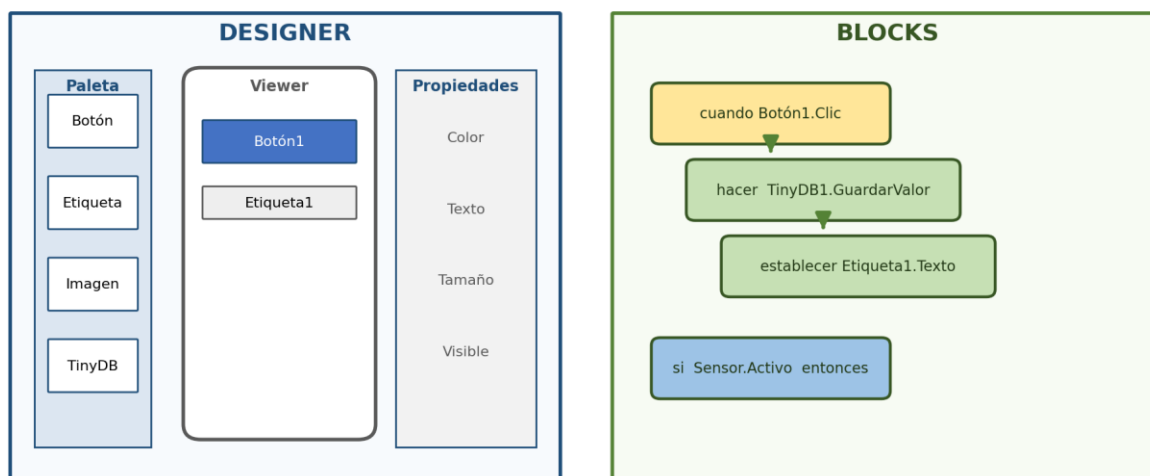
#### 3.1 ELEMENTOS PRINCIPALES DEL DESIGNER

El entorno de desarrollo incluye una paleta de componentes que agrupa más de 60 elementos disponibles, que van desde componentes de interfaz de usuario, diseño y medios, hasta sensores, conectividad y almacenamiento. Estos elementos se organizan visualmente en el área de visualización (Viewer), la cual representa la pantalla del dispositivo móvil y muestra la disposición de los componentes en tiempo real. Por otro lado, los componentes no visibles, como bases de datos o sensores, aportan funcionalidad interna sin aparecer en la interfaz gráfica. Finalmente, el panel de propiedades permite configurar atributos específicos de cada componente seleccionado —como su color, tamaño o visibilidad—, mientras que el administrador de pantallas (Screens) posibilita la creación de múltiples pantallas dentro de un mismo proyecto, cada una con su propio diseño y conjunto de bloques.

### 4. ENTORNO BLOCKS

El editor de bloques es la segunda vista de trabajo y define el comportamiento lógico de la aplicación. En lugar de escribir instrucciones en lenguaje textual, el desarrollador ensambla piezas visuales similares a rompecabezas que representan eventos, acciones, estructuras de control, operadores lógicos y matemáticos y llamadas a métodos para cada componente agregado al diseñador.

## Los dos entornos de trabajo de App Inventor



### 4.1 ELEMENTOS PRINCIPALES DEL DESIGNER

El sistema de programación se compone de diversos tipos de bloques, comenzando por los bloques de eventos, que definen las acciones a ejecutar ante eventos concretos como "cuando Botón1.Clic". A estos se suman los bloques de control, que incluyen estructuras condicionales y bucles, los bloques de lógica para operadores booleanos y comparaciones, y los bloques de matemáticas para operaciones aritméticas y funciones trigonométricas. Asimismo, se dispone de bloques de texto para la manipulación y concatenación de cadenas, junto con los bloques de variables y procedimientos, que permiten declarar variables y crear funciones reutilizables. Por último, el sistema integra los bloques propios de cada componente, los cuales se generan automáticamente según los elementos colocados en el Designer para gestionar de forma directa sus propiedades y métodos específicos, como guardar o recuperar valores en una base de datos.

## 5. TINYDB: ALMACENAMIENTO PERSISTENTE DE DATOS

De forma predeterminada, cualquier variable o valor que se procese en una aplicación App Inventor se pierde cuando el usuario cierra la aplicación, porque las variables solo existen mientras la aplicación se está ejecutando. Para almacenar información entre ejecuciones, App Inventor proporciona un componente invisible llamado TinyDB, una base de datos local persistente basada en pares etiqueta-valor (similar al concepto clave-valor utilizado en muchas bases de datos NoSQL).

## Funcionamiento de TinyDB: pares Etiqueta → Valor



### 5.1 ELEMENTOS PRINCIPALES DEL DESIGNER

El manejo de datos locales en la plataforma se realiza principalmente a través de TinyDB, donde para guardar un dato se utiliza el bloque TinyDB1.GuardarValor vinculando el contenido a una etiqueta (Tag) de texto identificativa, mientras que para recuperar un dato se emplea el bloque TinyDB1.ObtenerValor, especificando dicha etiqueta y un valor por defecto en caso de que esta aún no exista. Este componente garantiza la persistencia de la información, la cual permanece disponible en el almacenamiento interno del dispositivo incluso tras cerrar la aplicación o reiniciar el móvil. Además, este sistema tiene un alcance por aplicación, lo que impide compartir datos de TinyDB entre aplicaciones distintas, y mantiene un único almacén por proyecto, de modo que si se usan varios componentes TinyDB, todos accederán al mismo espacio, recomendándose el uso de prefijos o etiquetas diferentes para simular tablas independientes. Es importante tener en cuenta que al realizar pruebas con AI Companion, diferentes proyectos abiertos pueden compartir temporalmente la misma base de datos, un comportamiento que no ocurre una vez que la aplicación está compilada e instalada de forma definitiva como APK. Por último, para escenarios que requieran compartir información entre múltiples usuarios o dispositivos, App Inventor ofrece alternativas de almacenamiento en la nube como CloudDB y FirebaseDB, las cuales operan de manera muy similar a TinyDB pero guardan los datos en un servidor externo.

## 6. AI COMPANION: PRUEBA EN TIEMPO REAL DE LA APLICACIÓN

MIT AI2 Companion es una aplicación gratuita disponible en Google Play y App Store que tiene como objetivo permitir a un desarrollador ver y probar su aplicación en un dispositivo real (o emulador) mientras aún se está compilando en el navegador, sin tener que compilar o instalar un archivo APK primero.

### 6.1 PARA QUÉ SIRVE

Esta herramienta permite la denominada "prueba en vivo" (live testing), lo que significa que cualquier cambio realizado en el Designer o en los Blocks se refleja de manera casi inmediata en el dispositivo conectado. Al evitar el proceso de compilar y reinstalar la aplicación cada vez que se realiza un ajuste, se acelera de forma considerable el ciclo de desarrollo y depuración. Además, ofrece la gran ventaja de permitir probar el uso de sensores y hardware real del dispositivo —como la cámara, el GPS, el acelerómetro o el Bluetooth— durante el propio desarrollo, algo que un emulador de escritorio no siempre puede simular con total fidelidad.

## 6.2 CÓMO SE UTILIZA

Para establecer este tipo de conexión, primero se instala la aplicación MIT AI2 Companion en el dispositivo móvil desde la tienda de aplicaciones correspondiente y, de forma simultánea, en el proyecto de App Inventor se selecciona la opción "Conectar" y luego "AI Companion", lo cual genera un código QR y un código alfanumérico de seis caracteres. Acto seguido, se abre la aplicación desde el móvil y se escanea el código QR, o bien se introduce manualmente si no se dispone de cámara, asegurando siempre que tanto el ordenador como el dispositivo móvil estén conectados a la misma red Wi-Fi para hacer posible la comunicación entre ambos. Una vez establecida la conexión, la aplicación se ejecuta dentro de Companion y se actualiza automáticamente conforme se modifica el proyecto en el navegador; no obstante, como alternativas a este método, App Inventor también permite probar los proyectos mediante un emulador de escritorio —el cual requiere ejecutar el proceso auxiliar aiStarter— o a través de una conexión directa por cable USB.

## 7. VENTAJAS Y LIMITACIONES DE APP INVENTOR

### 7.1 VENTAJAS

Entre sus principales ventajas destaca una curva de aprendizaje muy baja que, al eliminar la sintaxis textual, resulta accesible para principiantes, estudiantes y personas sin formación previa en programación. Al ser una herramienta gratuita y basada en la nube, no requiere la instalación de un IDE pesado, ya que basta con un navegador web y una cuenta de Google para comenzar. Esto facilita un prototipado y desarrollo rápido, permitiendo construir aplicaciones funcionales en poco tiempo gracias a sus componentes prediseñados y a la prueba en vivo mediante AI Companion, lo cual acelera notablemente la depuración frente a los ciclos de compilación tradicionales. Asimismo, ofrece una amplia biblioteca de componentes que integra sensores, conectividad, multimedia y almacenamiento sin necesidad de recurrir a bibliotecas externas, todo bajo un enfoque educativo respaldado por el MIT que cuenta con abundante documentación, tutoriales y una comunidad activa a nivel mundial.

### 7.2 LIMITACIONES

Por otra parte, la plataforma presenta limitaciones importantes, comenzando por el hecho de que está diseñada para aplicaciones de complejidad limitada, por lo que no resulta adecuada para proyectos que requieran lógica muy avanzada, procesamiento intensivo o gráficos 3D de alto rendimiento. Además, existe una clara dependencia de la conexión a internet, ya que tanto el desarrollo como la compilación requieren una conectividad estable en la nube, a lo que se suma una personalización visual restringida que limita el diseño a los estilos predefinidos de la plataforma. De igual forma, ofrece una escalabilidad reducida para proyectos de gran tamaño, pues la organización visual se vuelve difícil de mantener a medida que crece el número de bloques y pantallas, mientras que su compatibilidad con iOS todavía es limitada y no cubre todos los componentes de Android. Finalmente, se percibe una menor preparación para el mercado laboral profesional, debido a que el uso de bloques no sustituye el aprendizaje de lenguajes y frameworks estándar de la industria como Kotlin o Swift.

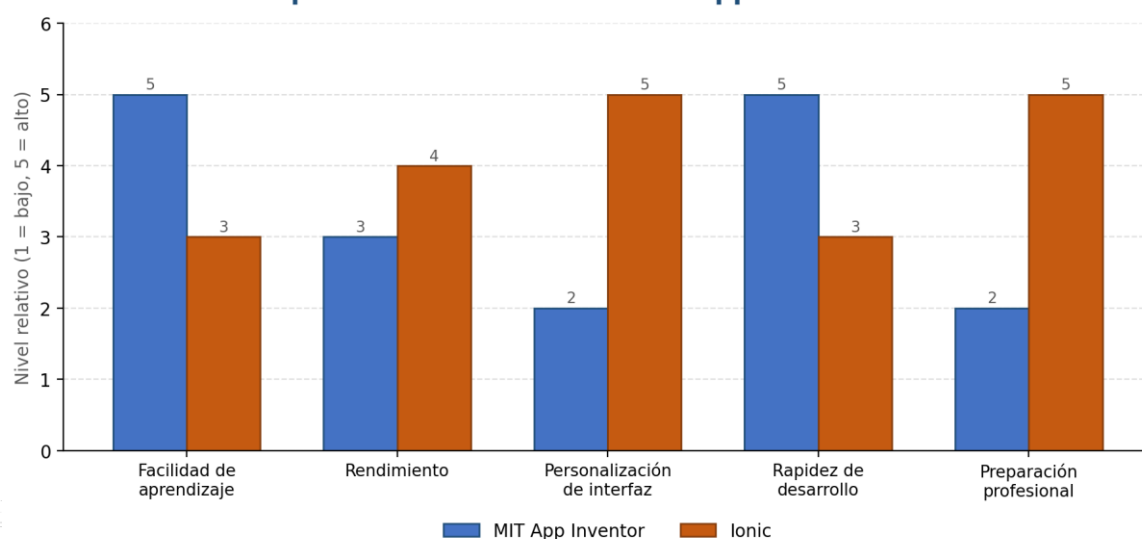
## 8. COMPARACIÓN CON IONIC

Ionic es un marco de código abierto que tiene como objetivo desarrollar aplicaciones móviles híbridas basadas en tecnologías web estándar (HTML, CSS y JavaScript/TypeScript) y complementadas con herramientas como Capacitor o Apache Cordova para acceder a la funcionalidad nativa del dispositivo. A diferencia de App Inventor, Ionic está diseñado para desarrolladores con conocimientos previos de programación web.

### 8.1 TABLA COMPARATIVA

| Criterio                       | MIT App Inventor                                                                                     | Ionic                                                                                                                       |
|--------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Lenguaje                       | Programación visual por bloques (Blockly); no requiere escribir código de texto.                     | HTML, CSS y JavaScript/TypeScript; se integra con frameworks como Angular, React o Vue.                                     |
| Curva de aprendizaje           | Muy baja; pensada para principiantes y fines educativos.                                             | Baja a moderada; requiere conocimientos previos de desarrollo web (HTML/CSS/JS).                                            |
| Rendimiento                    | Genera una app nativa de Android (o iOS), pero con posibilidades limitadas de optimización avanzada. | Rendimiento cercano al nativo gracias a Capacitor, aunque inferior al desarrollo nativo puro en tareas gráficas intensivas. |
| Publicación                    | Genera directamente un archivo APK/AAB desde la nube, listo para instalar o subir a Google Play.     | Requiere un proceso de compilación con Capacitor/Cordova y herramientas nativas (Android Studio/Xcode) antes de publicar.   |
| Personalización de la interfaz | Limitada a los componentes predefinidos de la plataforma.                                            | Amplia, mediante CSS y una extensa librería de componentes de interfaz adaptables a iOS y Android.                          |
| Público objetivo               | Estudiantes, principiantes y proyectos educativos o de prototipado rápido.                           | Desarrolladores web que buscan reutilizar sus habilidades para crear apps multiplataforma de nivel profesional.             |

**Comparación relativa entre MIT App Inventor e Ionic**



## 8.2 DIFERENCIAS CLAVE

A diferencia de App Inventor, que abstrae completamente el código detrás de bloques visuales, Ionic exige escribir y estructurar código en lenguajes de programación estándar de la web. Esta distinción se debe a que App Inventor está optimizado para el aprendizaje y el prototipado educativo, mientras que Ionic está orientado directamente a la producción de aplicaciones comerciales y de uso profesional en la industria. En consecuencia, el proceso de publicación también varía significativamente, ya que App Inventor ofrece un método más directo a través de su compilación en la nube, mientras que Ionic requiere un flujo de trabajo mucho más completo y riguroso con herramientas de compilación nativa.